FRED TALKS

10th May 2026

CONTENTS

LLM Evaluation: Practical Tips at Booking.com

GEORGE CHOULIARAS · 13 SEP 2025 · SOURCE

Lessons learned from 1 year of Judge-LLM Development

By

Zeno Belligoli

,

Antonio Castelli

,

George Chouliaras

The increasing adoption of Large Language Models (LLMs) across various industries has made evaluating LLM-powered applications (also called Generative AI applications) a critical necessity.

LLMs are pre-trained on a vast amount of text and can therefore be used to perform a wide range of tasks based on a natural language input called **prompt**. These tasks range from extracting information, answering complex questions, summarizing documents, generating creative content, translating languages, writing code etc.

However, unlike traditional Machine Learning (ML) models, the nature of LLMs brings inherent challenges that must be carefully considered:

- LLMs can **hallucinate**, i.e. can generate outputs which are factually incorrect or nonsensical while presenting them with high confidence
- LLMs can fail to **follow the instructions** specified in the prompt no matter how detailed they sound to a human reader
- Usage of LLMs has a **non-negligible cost** whether the model is open-source and served in-house or closed-source and accessed via a paid API.

To maximize the potential of Generative AI (GenAI) applications and mitigate the risks associated with them, we built a framework capable of thoroughly evaluating the performance of an LLM on a specific task in a nearly automated way. This framework is based on the concept of **LLM-as-judge**, i.e. on the usage of a more powerful LLM to evaluate the "target" LLM.

The LLM-as-a-judge framework

The main difference between the evaluation of traditional ML models and LLMs is that in most of the cases we are dealing with generative tasks for which either no single ground truth exists or is hard / expensive to obtain. When we ask an LLM to write a description of a property, or to summarize a long text, we don't really have an unambiguous reference that we can use for comparison. Moreover we can't get this at scale for different topics or subjects.

Therefore the measurement of text attributes like clarity, readability, toxicity or factual accuracy becomes more challenging. It would be possible, in theory, to employ human experts to review every generation and provide an estimation of the metric under study. However, this process would be time consuming and expensive to be **practically infeasible**.

The LLM-as-judge approach, requires human involvement **only once** (unless the production distribution changes), to carefully annotate the so-called **golden dataset** which must be large enough to be representative of the data distribution in production (the best practices for golden dataset creation are described in the next section).

Once we have the golden dataset with labels, we can prompt (or fine-tune) an LLM to replicate human judgement as much as possible. Once this is achieved, (e.g. when the agreement between the judge LLM and the human annotation has an accuracy above a certain threshold) the judge LLM can be employed to score the predictions of a second LLM (*target LLM*) on other datasets.

> This allows the continuous monitoring of the performance of a GenAI application in production in a scalable way with minimal human involvement.

Since the judge-LLM solves a classification task (e.g. is the text clear or not? is the text factually accurate or not?) getting golden labels is much easier. Hence, we can evaluate its performance using standard metrics like accuracy, f1-score, precision or recall, to monitor the performance of the target LLM. The process of judge-LLM development and usage is depicted in Figure 1.

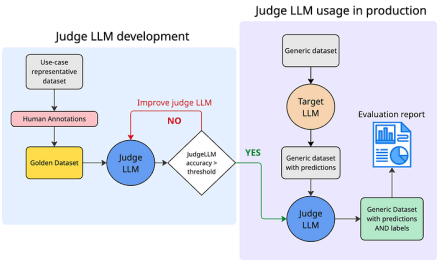

Figure 1: Judge-LLM development and deployment cycle. The left box (light-blue) shows the judge-LLM development phase. This phase always starts with a golden dataset with labels provided by a fully trusted source (e.g. human experts). A base LLM model is prompted or fine-tuned to imitate human label scoring. When the judge-LLM reaches a certain accuracy with respect to the golden labels, it can be deployed in production (right purple box) to score any other similar dataset, for example to evaluate the output of a target LLM. An evaluation report can be produced using the predictions of the target LLM and the label provided by the Judge LLM.

Golden Dataset Creation

The main aim of a golden dataset is to **accurately assess the ability** of a judge-LLM, to evaluate a production LLM in a particular evaluation metric. Due to this, a **high quality** golden dataset should meet the following criteria:

- It should represent the *production distribution*, since we want the Judge LLM evaluation to be as close to production as possible.

- It should include *golden* labels, e.g. labels with high quality. This can be proxied by high inter-annotator agreement among the annotators of the labels.

Low inter-annotator agreement usually indicates ambiguity in the metric definition and should be resolved by calibration among the annotators and by adding clarifications in the definition until the agreement is acceptable.

Since text attributes often do not have an objective and unique true value, achieving high-quality annotations is not a trivial task. To make fair comparisons between different versions of a GenAI application it is of utmost importance to have a **standardized annotation protocol** which is rigorously followed. Below we provide both a **basic** and an **advanced** annotation protocol for creating the golden dataset.

Basic Annotation Protocol

The basic annotation protocol can be used when only a single annotator is available.

1. Metric Definition

Define the metric definition with the business owner in the most unambiguous way possible.

- Write a **clear** definition of the metric as objective as possible. Prefer **binary** metrics or categorical metrics with a few classes, since LLMs struggle with continuous scores . If you're forced to use a continuous metric, consider binning into a few coarse ordinal values e.g. 'high', 'medium', 'low'.

- Write **annotation instructions**, using the metric definition. These are the instructions a human should follow in order to provide the ground truth label. Describe the task **clearly** and **objectively**. Avoid vague words (such as major, minor, a few). Provide scored examples and include edge cases. Specify what tools the annotators can and cannot use (Search engines, ChatGPT, etc).

2. Pilot Annotation

Collect a few examples (~50) and annotate them according to the aforementioned metric definition. The annotators should carefully follow the instructions provided, without bringing any previous knowledge or bias on the task. The annotators at this stage can be selected from business owners or from other technical people involved. The aim of this step is to ensure that we have a clear metric definition and that the annotations are aligned with it.

Once the annotations are done, it is recommended that they are **reviewed** by a pool of domain experts to assess if they align with the guidelines. If issues or disagreements are found, they should be resolved, the metric definition should be updated and the pilot annotation should be repeated until full consensus is reached.

3. Full Annotation by a single annotator

Once the annotation of the first small sample during the pilot annotation is finalized and reviewed, the examples can be shared to the annotator as guidance in order to annotate a full dataset. The full dataset should consist of **ideally 500- 1000 examples** at least. This is because part of the dataset will be used for tuning the judge-LLM (validation set) and another part as a holdout set in order to evaluate its performance.

4. Quality Review

A subject matter expert reviews a **representative** sample (~10%) of the full annotation. If a significant percentage (> 10%) of the labels in the sample are incorrect, annotation should be repeated. The quality thresholds can be adjusted according to the organization's needs and should be communicated and agreed upon with the annotators before the full annotation.

Advanced Annotation Protocol

The advanced annotation protocol requires more effort but also guarantees higher quality annotations than the basic one.

1. Metric Definition

Same as in the basic protocol.

2. Pilot Annotation

Same as in the basic protocol.

3. Full Annotation by multiple annotators

Instead of having a single human annotator providing the label for a given example, we have multiple human annotators (3+) providing a label for each example.
Depending on the use case, an aggregation function must be defined to obtain the final label. The aggregation function should take an array of labels and return a single one. In principle any

aggregation function can be used. If labels can be converted into integers and ordered any function like min, max, or average can be used. If labels have no orders (e.g. geographical location) majority vote can be used.

Examples where the annotators **disagree** can be treated in multiple ways, depending on what is more appropriate for the specific problem.

a) Add a "not sure" class: map to the "not sure" class all examples without 100% consensus

b) Add a weight < 1: For all examples without 100% consensus select the majority class among the annotated ones (a random one if there is no majority) and add a weight to the example equal to n_selected_class_votes / n_total_votes. This approach will penalize the more uncertain examples in the aggregated performance.

c) Discard the ambiguous examples: For some use cases it might make sense to simply remove all the examples where there is no 100% agreement.

4. Quality Review

Same as in the basic protocol.

How to develop a judge-LLM

Once the golden dataset is ready, you can develop the judge-LLM. Note that you can already do this step once the sample dataset from the pilot annotation is ready (with ~50 samples) to create a first version of the judge-LLM and score a few examples to identify any early issues. However for proper evaluation and tuning of the judge-LLM you need a large dataset of 500–1000 examples. In this article we share how to create a prompt-based judge-LLM for simplicity. You can always fine-tune a judge-LLM but this won't be covered here. To develop the judge-LLM you follow the **iterative process** detailed below, which is the standard process for manual prompt engineering.

1. **Split the golden dataset in validation and test sets:** A standard split is 50/50 but the final decision is on the model owner. The validation set is used to compute the prompt performance and to find error patterns. The test set is used to check for evaluating generalization and overfitting.

2. **Choose a strong LLM:** We typically select powerful models such as **GPT-4.1**, or **Claude 4.0 Sonnet** as the backbone for our judge-LLM. These models serve two roles:

a) As a sanity check: Given a high-quality dataset and a well-defined task, a strong LLM should perform well if the prompt is reasonable.

b) To estimate an upper bound on achievable performance. While this is not a strict bound (there are cases where smaller models with better prompts outperform stronger ones) it provides a useful reference point.

3. Write the prompt of the judge-LLM: Use the metric definition and include scoring instructions and output format (can be json/string/else). Few shot examples can also be included but ensure that no examples from the golden dataset are provided to avoid overfitting. We also normally add chain-of-thought prompting (CoT) to promote reasoning and explainability. To build the judge-LLM you can use any framework that supports custom LLM-based metrics, such as DeepEval's G-Eval metric or Arize Phoenix.

4. Evaluate the performance on the validation set: The evaluation metric depends on the type of the task and the class imbalance. For classification tasks we generally recommend macro f1-score while for continuous metric you can use mean squared error.

5. Perform error analysis and update the prompt
We analyze the model's mistakes on the validation set to identify recurring issues or edge cases. Based on these insights, we revise the prompt to better align it with the desired behavior. We then repeat steps 3, 4 and 5. Typically, we are satisfied once we reach a value of the evaluation metric above a certain threshold agreed with the business stakeholders.

6. Evaluate the performance on the test set: This will give the unbiased performance score of the judge-LLM. It is not recommended to change the prompt in order to improve the performance on the test set, as this might end up in overfitting on this dataset.

After prompt engineering is completed with a strong model, we repeat the same process using a **weaker (and more cost-efficient) model**, aiming to match the performance of the stronger version as closely as possible. In practice, we use the **strong-model judge-LLM during AI system development**, where quality is critical, and **the weaker-model version for large-scale monitoring** of system performance in production environments. An example of such monitoring is provided in Figure 2.

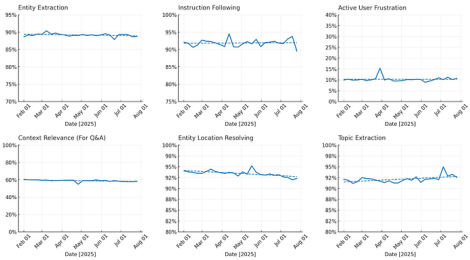

Figure 2: The figure shows an example dashboard used to monitor the quality of an LLM application, based on judge-LLM metrics. Metric values can also be combined to get aggregated scores. In this particular example we show the trends for Entity Extraction accuracy, LLM-instruction-following accuracy, frustration of users, context relevance, LLM-location-resolving accuracy and LLM-topic-conversation-extraction accuracy. From the plot it is immediate to quickly spot issues connected to instruction-following and topic-extraction allowing for prompt mitigation actions. An automated system is set-up to alert the application owners whenever an anomaly is detected for any of the relevant metrics.

Future directions

Pointwise vs. Comparative Judges

The vast majority of our judge-LLMs are **pointwise judges**, which assign an absolute score to each response independently. Pointwise judges are particularly useful because they can serve **dual purposes**:

- Ranking system outputs during development.
- Monitoring performance in production, where only one system's response is typically available.

However, **comparative (pair-wise)** judges (which compare two responses and select the better one) tend to offer **stronger ranking signals**. It is often easier for a model to decide which of two answers is better than to assign calibrated absolute scores. For this reason, comparative judges can be more effective during the AI system development phase. We are actively exploring methods to **develop both pointwise and comparative judges with minimal extra overhead**, ideally reusing most of the dataset and prompt engineering efforts.

Automatic judge-LLM development

Prompt engineering remains a **manual, iterative, and time-intensive process**, typically taking anywhere from **one day to a full week** depending on the complexity of the task. To streamline this, we have developed an **automated prompt engineering pipeline** inspired by DeepMind's OPRO that, in essence, automates **steps 3–5** of the process. At the end of the process it is always important that the human inspects the final prompt to ensure that it doesn't contain use case specific examples or rules that might undermine generalizability.

Another bottleneck in the judge-LLM development process is that of data annotation. Annotating a good dataset requires substantial effort from multiple annotators and can take from some days to weeks, depending on the complexity of the task. In this context, one can leverage the ability of LLMs to generate realistic text to create **synthetic data**. This is particularly helpful and achievable for classification tasks, whereby the LLM only needs to generate the input X, conditioned on a certain class Y. Substantial challenges in this area lie in the generation of diverse and realistic text and conversations and in the quality evaluation of the synthetic data, that is, how do we know that our synthetic data is realistic, diverse and of high-quality? We are actively working on this area and are looking forward to sharing more with you in a future blog post.

Evaluating LLM agents

We did not cover LLM-based **agent evaluation** in this article as it is a topic on its own. This is because evaluating agents often requires assessing long-term goal completion, reasoning over multi-step interactions, handling complex tool use and effective use of planning and memory. These are challenges that go beyond the evaluation of isolated model outputs and typically involve different methodologies, benchmarks, and infrastructure. We will cover this topic in a separate blog post.

Key takeaways

Evaluating generative tasks is inherently complex due to the absence of clear ground truth data. The LLM-as-a-judge framework offers an efficient method to scale this evaluation process in an automated way. A high-quality judge-LLM is built upon a "golden dataset" with reliable labels, which necessitates a rigorous annotation protocol; we have outlined both a basic and an advanced version to achieve this. This judge-LLM can then be optimized through manual, iterative prompt engineering or via automated methods. Future directions in this field include creating golden datasets with synthetic data and developing evaluation methods for LLM-based agents.

Eight years of wanting, three months of building with AI

LALIT MAGANTI · 12 APR 2026 · SOURCE

For eight years, I've wanted a high-quality set of devtools for working with SQLite. Given how important SQLite is to the industry[1], I've long been puzzled that no one has invested in building a really good developer experience for it[2].

A couple of weeks ago, after ~250 hours of effort over three months[3] on evenings, weekends, and vacation days, I finally released syntaqlite (GitHub), fulfilling this long-held wish. And I believe the main reason this happened was because of AI coding agents[4].

Of course, there's no shortage of posts claiming that AI one-shot their project or pushing back and declaring that AI is all slop. I'm going to take a very different approach and, instead, systematically break down my experience building syntaqlite with AI, both where it helped *and* where it was detrimental.

I'll do this while contextualizing the project and my background so you can independently assess how generalizable this experience was. And whenever I make a claim, I'll try to back it up with evidence from my project journal, coding transcripts, or commit history[5].

In my work on Perfetto, I maintain a SQLite-based language for querying performance traces called PerfettoSQL. It's basically the same as SQLite but with a few extensions to make the trace querying experience better. There are ~100K lines of PerfettoSQL internally in Google and it's used by a wide range of teams.

Having a language which gets traction means your users also start expecting things like formatters, linters, and editor extensions. I'd hoped that we could adapt some SQLite tools from open source but the more I looked into it, the more disappointed I was. What I found either wasn't reliable enough, fast enough[6], or flexible enough to adapt to PerfettoSQL. There was clearly an opportunity to build something from scratch, but it was never the "most important thing we could work on". We've been reluctantly making do with the tools out there but always wishing for better.

On the other hand, there *was* the option to do something in my spare time. I had built lots of open source projects in my teens[7] but this had faded away during university when I felt that I just didn't have the motivation anymore. Being a maintainer is much more than just "throwing the code out there" and seeing what happens. It's triaging bugs, investigating crashes, writing documentation, building a community, and, most importantly, having a direction for the project.

But the itch of open source (specifically freedom to work on what I wanted while helping others) had never gone away. The SQLite devtools project was eternally in my mind as "something I'd like to work on". But there was another reason why I kept putting it off: it sits at the intersection of being both hard *and* tedious.

What makes it hard and tedious

If I was going to invest my personal time working on this project, I didn't want to build something that only helped Perfetto: I wanted to make it work for *any* SQLite user out there[8]. And this means parsing SQL *exactly* like SQLite.

The heart of any language-oriented devtool is the parser. This is responsible for turning the source code into a "parse tree" which acts as the central data structure anything else is built on top of. If your parser isn't accurate, then your formatters and linters will inevitably inherit those inaccuracies; many of the tools I found suffered from having parsers which approximated the SQLite language rather than representing it precisely.

Unfortunately, unlike many other languages, SQLite has no formal specification describing how it should be parsed. It doesn't expose a stable API for its parser either. In fact, quite uniquely, in its implementation it doesn't even build a parse tree at all[9]! The only reasonable approach left in my opinion is to carefully extract the relevant parts of SQLite's source code and adapt it to build the parser I wanted[10].

This means getting into the weeds of SQLite source code, a fiendishly difficult codebase to understand. The whole project is written in C in an incredibly dense style; I've spent days just understanding the virtual table API[11] and implementation. Trying to grasp the full parser stack was daunting.

There's also the fact that there are >400 rules in SQLite which capture the full surface area of its language. I'd have to specify in each of these "grammar rules" how that part of the syntax maps to the matching node in the parse tree. It's extremely repetitive work; each rule is similar to all the ones around it but also, by definition, different.

And it's not just the rules but also coming up with and writing tests to make sure it's correct, debugging if something is wrong, triaging and fixing the inevitable bugs people filed when I got something wrong…

For years, this was where the idea died. Too hard for a side project[12], too tedious to sustain motivation, too risky to invest months into something that might not work.

I've been using coding agents since early 2025 (Aider, Roo Code, then Claude Code since July) and they'd definitely been useful but never something I felt I could trust a serious project to. But towards the end of 2025, the models seemed to make a significant step forward in quality[13]. At the same time, I kept hitting problems in Perfetto which would have been trivially solved by having a reliable parser. Each workaround left the same thought in the back of my mind: maybe it's finally time to build it for real.

I got some space to think and reflect over Christmas and decided to really stress test the most maximalist version of AI: could I vibe-code the whole thing using just Claude Code on the Max plan (£200/month)?

Through most of January, I iterated, acting as semi-technical manager and delegating almost all the design and all the implementation to Claude. Functionally, I ended up in a reasonable place: a parser in C extracted from SQLite sources using a bunch of Python scripts, a formatter built on top, support for both the SQLite language and the PerfettoSQL extensions, all exposed in a web playground.

But when I reviewed the codebase in detail in late January, the downside was obvious: the codebase was complete spaghetti[14]. I didn't understand large parts of the Python source extraction pipeline, functions were scattered in random files without a clear shape, and a few files had grown to several thousand lines. It was *extremely* fragile; it solved the immediate problem *but* it was never going to cope with my larger vision, never mind integrating it into the Perfetto tools. The saving grace was that it had proved the approach was viable and generated more than 500 tests, many of which I felt I could reuse.

I decided to throw away everything and start from scratch while also switching most of the codebase to Rust[15]. I could see that C was going to make it difficult to build the higher level components like the validator and the language server implementation. And as a bonus, it would also let me use the same language for both the extraction and runtime instead of splitting it across C and Python.

More importantly, I completely changed my role in the project. I took ownership of all decisions[16] and used it more as "autocomplete on steroids" inside a much tighter process: opinionated design upfront, reviewing every change thoroughly, fixing problems eagerly as I spotted them, and investing in scaffolding (like linting, validation, and non-trivial testing[17]) to check AI output automatically.

The core features came together through February and the final stretch (upstream test validation, editor extensions, packaging, docs) led to a 0.1 launch in mid-March.

But in my opinion, this timeline is the least interesting part of this story. What I really want to talk about is what wouldn't have happened without AI and also the toll it took on me as I used it.

AI is why this project exists, and why it's as complete as it is

I've written in the past about how one of my biggest weaknesses as a software engineer is my tendency to procrastinate when facing a big new project. Though I didn't realize it at the time, it could not have applied more perfectly to building syntaqlite.

AI basically let me put aside all my doubts on technical calls, my uncertainty of building the right thing and my reluctance to get started by giving me very concrete problems to work on. Instead of "I need to understand how SQLite's parsing works", it was "I need to get AI to suggest an approach for me so I can tear it up and build something better"[18]. I work so much better with concrete prototypes to play with and code to look at than endlessly thinking about designs in my head, and AI lets me get to that point at a pace I could not have dreamed about before. Once I took the first step, every step after that was so much easier.

AI turned out to be better than me at the act of writing code itself, assuming that code is obvious. If I can break a problem down to "write a function with this behaviour and parameters" or "write a class matching this interface," AI will build it faster than I would and, crucially, in a style that might well be more intuitive to a future reader. It documents things I'd skip, lays out code consistently with the rest of the project, and sticks to what you might call the "standard dialect" of whatever language you're working in[19].

That standardness is a double-edged sword. For the vast majority of code in any project, standard is exactly what you want: predictable, readable, unsurprising. But every project has pieces that are its edge, the parts where the value comes from doing something non-obvious. For

syntaqlite, that was the extraction pipeline and the parser architecture. AI's instinct to normalize was actively harmful there, and those were the parts I had to design in depth and often resorted to just writing myself.

But here's the flip side: the same speed that makes AI great at obvious code also makes it great at refactoring. If you're using AI to generate code at industrial scale, you *have* to refactor constantly and continuously[20]. If you don't, things immediately get out of hand. This was the central lesson of the vibe-coding month: I didn't refactor enough, the codebase became something I couldn't reason about, and I had to throw it all away. In the rewrite, refactoring became the core of my workflow. After every large batch of generated code, I'd step back and ask "is this ugly?" Sometimes AI could clean it up. Other times there was a large-scale abstraction that AI couldn't see but I could; I'd give it the direction and let it execute[21]. If you have taste, the cost of a wrong approach drops dramatically because you can restructure quickly[22].

Of all the ways I used AI, research had by far the highest ratio of value delivered to time spent.

I've worked with interpreters and parsers before but I had never heard of Wadler-Lindig pretty printing[23]. When I needed to build the formatter, AI gave me a concrete and actionable lesson from a point of view I could understand and pointed me to the papers to learn more. I could have found this myself eventually, but AI compressed what might have been a day or two of reading into a focused conversation where I could ask "but why does this work?" until I actually got it.

This extended to entire domains I'd never worked in. I have deep C++ and Android performance expertise but had barely touched Rust tooling or editor extension APIs. With AI, it wasn't a problem: the fundamentals are the same, the terminology is similar, and AI bridges the gap[24]. The VS Code extension would have taken me a day or two of learning the API before I could even start. With AI, I had a working extension within an hour.

It was also invaluable for reacquainting myself with parts of the project I hadn't looked at for a few days[25]. I could control how deep to go: "tell me about this component" for a surface-level refresher, "give me a detailed linear walkthrough" for a deeper dive, "audit unsafe usages in this repo" to go hunting for problems. When you're context switching a lot, you lose context fast. AI let me reacquire it on demand.

Beyond making the project exist at all, AI is also the reason it shipped as complete as it did. Every open source project has a long tail of features that are important but not critical: the things you know theoretically how to do but keep deprioritizing because the core work is more

pressing. For syntaqlite, that list was long: editor extensions, Python bindings, a WASM playground, a docs site, packaging for multiple ecosystems[26]. AI made these cheap enough that skipping them felt like the wrong trade-off.

It also freed up mental energy for UX[27]. Instead of spending all my time on implementation, I could think about what a user's first experience should feel like: what error messages would actually help them fix their SQL, how the formatter output should look by default, whether the CLI flags were intuitive. These are the things that separate a tool people try once from one they keep using, and AI gave me the headroom to care about them. Without AI, I would have built something much smaller, probably no editor extensions or docs site. AI didn't just make the same project faster. It changed what the project *was*.

There's an uncomfortable parallel between using AI coding tools and playing slot machines[28]. You send a prompt, wait, and either get something great or something useless. I found myself up late at night wanting to do "just one more prompt," constantly trying AI just to see what would happen even when I knew it probably wouldn't work. The sunk cost fallacy kicked in too: I'd keep at it even in tasks it was clearly ill-suited for, telling myself "maybe if I phrase it differently this time."

The tiredness feedback loop made it worse[29]. When I had energy, I could write precise, well-scoped prompts and be genuinely productive. But when I was tired, my prompts became vague, the output got worse, and I'd try again, getting more tired in the process. In these cases, AI was probably slower than just implementing something myself, but it was too hard to break out of the loop[30].

Several times during the project, I lost my mental model of the codebase[31]. Not the overall architecture or how things fitted together. But the day-to-day details of what lived where, which functions called which, the small decisions that accumulate into a working system. When that happened, surprising issues would appear and I'd find myself at a total loss to understand what was going wrong. I hated that feeling.

The deeper problem was that losing touch created a communication breakdown[32]. When you don't have the mental thread of what's going on, it becomes impossible to communicate meaningfully with the agent. Every exchange gets longer and more verbose. Instead of "change FooClass to do X," you end up saying "change the thing which does Bar to do X". Then the agent has to figure out what Bar is, how that maps to FooClass, and sometimes it gets it wrong[33]. It's exactly the same complaint engineers have always had about managers who don't understand the code asking for fanciful or impossible things. Except now you've become that manager.

The fix was deliberate: I made it a habit to read through the code immediately after it was implemented and actively engage to see “how would I have done this differently?”.

Of course, in some sense all of the above is also true of code I wrote a few months ago (hence the sentiment that AI code is legacy code), but AI makes the drift happen faster because you’re not building the same muscle memory that comes from originally typing it out.

There were some other problems I only discovered incrementally over the three months.

I found that AI made me procrastinate on key design decisions[34]. Because refactoring was cheap, I could always say “I’ll deal with this later.” And because AI could refactor at the same industrial scale it generated code, the cost of deferring felt low. But it wasn’t: deferring decisions corroded my ability to think clearly because the codebase stayed confusing in the meantime. The vibe-coding month was the most extreme version of this. Yes, I understood the problem, but if I had been more disciplined about making hard design calls earlier, I could have converged on the right architecture much faster.

Tests created a similar false comfort[35]. Having 500+ tests felt reassuring, and AI made it easy to generate more. But neither humans nor AI are creative enough to foresee every edge case you’ll hit in the future; there are several times in the vibe-coding phase where I’d come up with a test case and realise the design of some component was completely wrong and needed to be totally reworked. This was a significant contributor to my lack of trust and the decision to scrap everything and start from scratch.

Basically, I learned that the “normal rules” of software still apply in the AI age: if you don’t have a fundamental foundation (clear architecture, well-defined boundaries) you’ll be left eternally chasing bugs as they appear.

Something I kept coming back to was how little AI understood about the passage of time[36]. It sees a codebase in a certain state but doesn’t *feel* time the way humans do. I can tell you what it feels like to use an API, how it evolved over months or years, why certain decisions were made and later reversed.

The natural problem from this lack of understanding is that you either make the same mistakes you made in the past and have to relearn the lessons *or* you fall into new traps which were successfully avoided the first time, slowing you down in the long run. In my opinion, this is a similar problem to why losing a high-quality senior engineer hurts a team so much: they carry history and context that doesn’t exist anywhere else and act as a guide for others around them.

In theory, you can try to preserve this context by keeping specs and docs up to date. But there's a reason we didn't do this before AI: capturing implicit design decisions exhaustively is incredibly expensive and time-consuming to write down. AI can help draft these docs, but because there's no way to automatically verify that it accurately captured what matters, a human still has to manually audit the result. And that's still time-consuming.

There's also the context pollution problem. You never know when a design note about API A will echo in API B. Consistency is a huge part of what makes codebases work, and for that you don't just need context about what you're working on right now but also about other things which were designed in a similar way. Deciding what's relevant requires exactly the kind of judgement that institutional knowledge provides in the first place.

Reflecting on the above, the pattern of when AI helped and when it hurt was fairly consistent.

When I was working on something I already understood deeply, AI was excellent. I could review its output instantly, catch mistakes before they landed and move at a pace I'd never have managed alone. The parser rule generation is the clearest example[37]: I knew exactly what each rule should produce, so I could review AI's output within a minute or two and iterate fast.

When I was working on something I could describe but didn't yet know, AI was good but required more care. Learning Wadler-Lindig for the formatter was like this: I could articulate what I wanted, evaluate whether the output was heading in the right direction, and learn from what AI explained. But I had to stay engaged and couldn't just accept what it gave me.

When I was working on something where I didn't even know what I wanted, AI was somewhere between unhelpful and harmful. The architecture of the project was the clearest case: I spent weeks in the early days following AI down dead ends, exploring designs that felt productive in the moment but collapsed under scrutiny. In hindsight, I have to wonder if it would have been faster just thinking it through without AI in the loop at all.

But expertise alone isn't enough. Even when I understood a problem deeply, AI still struggled if the task had no objectively checkable answer[38]. Implementation has a right answer, at least at a local level: the code compiles, the tests pass, the output matches what you asked for. Design doesn't. We're still arguing about OOP decades after it first took off.

Concretely, I found that designing the public API of syntaqlite was where this hit home the hardest. I spent several days in early March doing nothing but API refactoring, manually fixing things any experienced engineer would have instinctively avoided but AI made a total mess of. There's no test or objective metric for "is this API pleasant to use" and "will this API help users solve the problems they have" and that's exactly why the coding agents did *so badly* at it.

This takes me back to the days I was obsessed with physics and, specifically, relativity. The laws of physics look simple and Newtonian in any small local area, but zoom out and spacetime curves in ways you can't predict from the local picture alone. Code is the same: at the level of a function or a class, there's usually a clear right answer, and AI is excellent there. But architecture is what happens when all those local pieces interact, and you can't get good global behaviour by stitching together locally correct components.

Knowing where you are on these axes at any given moment is, I think, the core skill of working with AI effectively.

Eight years is a long time to carry a project in your head. Seeing these SQLite tools actually exist and function after only three months of work is a massive win, and I'm fully aware they wouldn't be here without AI.

But the process wasn't the clean, linear success story people usually post. I lost an entire month to vibe-coding. I fell into the trap of managing a codebase I didn't actually understand, and I paid for that with a total rewrite.

The takeaway for me is simple: AI is an incredible force multiplier for implementation, but it's a dangerous substitute for design. It's brilliant at giving you the right answer to a specific technical question, but it has no sense of history, taste, or how a human will actually feel using your API. If you rely on it for the "soul" of your software, you'll just end up hitting a wall faster than you ever have before.

What I'd like to see more of from others is exactly what I've tried to do here: honest, detailed accounts of building real software with these tools; not weekend toys or one-off scripts but the kind of software that has to survive contact with users, bug reports, and your own changing mind.

2.1.138

CHANGELOG · 09 MAY 2026 · SOURCE

- Internal fixes

2.1.137

CHANGELOG · 09 MAY 2026 · SOURCE

- [VSCode] Fixed extension failing to activate on Windows